

分布式缓存

一、 分布式缓存概述

引言

在现代软件架构中，性能优化和系统可扩展性是两个至关重要的方面。在 Dev Story 系列视频中深入讨论了这些主题，特别强调了缓存技术在提升系统性能和减少延迟方面的关键作用。本报告将总结相关内容，并提供关于缓存的详细分析和实施建议。

系统性能提升方法

水平扩展与分层架构

- 水平扩展：通过增加更多的服务器节点来处理更多的并发请求。
- 分层架构：通过将系统分为多个功能层次，每个层次专注于特定的任务，从而提高系统的组织和管理效率。
- 限制因素：如受限的 DRM（数字版权管理），尽管系统能够处理每秒数百万的请求，但延迟仍然较大。

为了解决这些性能瓶颈，缓存技术被推荐作为一种有效的优化手段。

缓存的类型与应用

分布式缓存是一种在软件架构中用于提高性能、可扩展性和可靠性的高级缓存机制。通过在多个物理服务器或节点上存储数据的副本，分布式缓存允许系统更快地响应用户请求，同时减轻后端数据库的负载。

分布式缓存在现代分布式系统和大规模应用中非常常见，例如在电商网站、社交媒体平台和内容分发网络（CDN）中。

1、CDN（内容分发网络）

作用：缓存静态资源（如图片和视频），避免用户重复下载相同内容。

优势：减轻源服务器负担，提高用户访问速度。

2、内存或本地缓存

作用：在服务器上缓存频繁访问的数据。

优势：减少数据查询时间，加快响应速度。

3、 分布式缓存

工具：Redis、Memcached。

作用：在访问数据库之前缓存数据，减少数据库查询次数。

优势：提高系统性能，减轻数据库负载。

分布式缓存的工作原理

1、 数据分片

将缓存数据根据某种策略（例如一致性哈希）分片，分配到多个缓存节点上。这些节点可以是不同的物理服务器或虚拟机。

2、 数据存取

当需要存取数据时，系统通过哈希函数或其他映射机制确定数据存储的位置，直接访问对应的缓存节点。

3、 一致性和复制

为了提高可靠性，分布式缓存系统通常会在多个节点之间复制数据副本，确保在某些节点故障时数据仍然可用。

4、 缓存失效和更新

分布式缓存需要处理缓存失效和数据更新的问题。常见的方法包括设置缓存数据的 TTL (Time-to-Live)，或者在数据源发生变化时主动更新缓存。

分布式缓存的优势

1、 性能提升

缓存数据存储在内​​存中，访问速度比磁盘存储的数据库快得多，可以显著减少数据访问时间。

2、 减轻数据库负载

通过将频繁访问的数据缓存起来，减少对数据库的直接访问，降低数据库的负载，提升整体系统性能。

3、 可扩展性

分布式缓存可以通过增加缓存节点来扩展容量和性能，处理更大规模的并发访问和数据存储需求。

4、高可用性

通过数据复制和故障转移机制，分布式缓存可以在部分节点失效的情况下继续提供服务，提高系统的可用性。

缓存的复杂性与挑战

1、缓存验证

困难性：计算机科学中缓存验证被认为是最困难的问题之一，确保缓存数据的正确性和一致性非常复杂。

2、更新延迟

问题：缓存数据没有及时失效可能导致过时数据的显示。

解决方案：设计有效的缓存失效策略，确保数据及时更新。

3、缓存未命中

问题：在多服务器环境中，用户请求可能被重定向到没有缓存其数据的服务器，导致必须重新计算和缓存数据。

解决方案：实现会话粘性或全局缓存管理。

4、设计难度

问题：缓存策略设计非常复杂，特别是在多层缓存架构中。

解决方案：深入分析应用需求，设计合理的缓存层次和失效策略。

缓存作为优化技术

1、实施时机

缓存通常在系统开发的后期阶段添加，作为优化技术，而非初期必须实现的功能。

2、架构设计考虑

尽管缓存可以后期添加，但在架构设计时应考虑到未来的缓存使用，以便于系统的持续优化和扩展。

常见的分布式缓存技术和工具

1、Redis

Redis 提供丰富的数据结构支持和持久化功能，是一种内存数据结构存储系统，

支持多种数据结构（如字符串、哈希、列表、集合等），并提供了主从复制和集群功能，适用于构建分布式缓存。

2、Memcached

Memcached 专注于高速键值对缓存，是一种高性能的分布式内存缓存系统，主要用于缓存数据库查询结果，以加速动态 Web 应用程序的访问。

总结：

分布式缓存是一种强大的工具，能够显著提升系统性能和可扩展性。然而，缓存的设计和实施需要精心规划，以确保系统的稳定性和数据的准确性。通过合理的缓存策略，系统可以在高并发请求下保持高效运行，为用户提供快速、可靠的服务。

二、 缓存的雪崩和击穿

缓存雪崩和缓存击穿是分布式缓存系统中常见的问题，它们会在特定情况下发生，对系统性能和稳定性产生显著影响。以下是这些情况的发生条件、表现以及预防和补救措施的详细说明。

缓存雪崩

1、发生情况

缓存雪崩发生在大量缓存数据同时失效的情况下。这通常是由于缓存服务器重启、集群故障或者缓存数据设置了相同的过期时间，导致大量请求同时打到后端数据库。

举例：

- **同时失效：**就像超市里所有冰激凌同时融化了，所有顾客都只能去仓库重新取新的冰激凌。这种情况在缓存系统中就是缓存雪崩。

- **服务器重启或故障：**如果超市的所有收银员同时下班，所有顾客只能排队去仓库取货，也会造成大混乱。

2、表现

- 1) 短时间内大量请求涌入数据库，导致数据库负载骤增。
- 2) 系统响应时间变长，甚至可能导致服务崩溃。
- 3) 用户体验显著下降，可能出现服务不可用的情况。

举例：

- **高峰期堵塞：**系统会突然变得非常慢，因为所有的请求都直接打到数据库上（相当于所有顾客都去仓库取货）。
- **可能宕机：**数据库负载过大，可能会崩溃（超市仓库也会不堪重负）。

3、预防和补救措施

- 1) **设置不同的缓存过期时间：**通过在设置缓存过期时间时加上随机数，避免所有缓存数据同时失效。例如：`cacheTimeout = baseTimeout + random.nextInt(1000)`
- 2) **使用多级缓存：**在本地缓存和分布式缓存之间构建多级缓存体系，减轻后端数据库的压力。
- 3) **限流与降级：**在缓存失效期间，对进入数据库的请求进行限流，保护数据库不被压垮。同时可以返回默认数据或部分缓存数据作为降级措施。
- 4) **缓存预热：**在系统启动或重新部署时，提前加载部分热点数据到缓存中，避免缓存数据在短时间内大量失效。
- 5) **集群化部署：**使用缓存集群，分散缓存数据的存储位置和失效时间，防止单点故障导致缓存雪崩。

以下是预防和补救措施

- 1) **随机过期时间：**设置不同的过期时间（比如，冰激凌有的1天后过期，有的2天后过期），避免同时失效。

- 2) **多级缓存：**在超市内部设置几个小仓库，先从小仓库取货，减轻大仓库的压力。
- 3) **限流与降级：**限制每次进入仓库的人数，避免太多人同时进入。降级措施可以是提供临时替代商品。
- 4) **缓存预热：**在超市开门前，提前补货（预先加载热点数据到缓存中）。
- 5) **集群化部署：**多开几家分店，各自管理自己的库存，避免集中在一家超市。

理解场景：

假设你经营一个超市，里面有一批冰激凌，设置了一个固定的过期时间（比如 1 小时）。结果过了 1 小时，所有冰激凌同时过期，所有顾客都不得不去仓库取新的冰激凌，导致仓库被挤爆，速度变得非常慢。为了避免这种情况，你可以让冰激凌有不同的过期时间，比如 1 小时、1 小时 10 分钟、1 小时 20 分钟，这样就不会同时过期，避免仓库被挤爆。

缓存击穿

1、发生情况

缓存击穿是指某些热点数据（特别是一些高并发访问的数据）在缓存失效后，大量请求同时访问该数据，从而直接打到数据库上。

举例：

- **热点数据失效：**就像某个非常受欢迎的明星签名商品突然下架了，所有粉丝都直接跑去仓库取货。
- **并发请求：**很多人同时访问同一个热门商品，结果缓存失效了，大量请求涌向数据库。

2、表现

- 1) 对特定热点数据的请求直接涌入数据库，导致数据库负载骤增。
- 2) 系统响应时间变长，影响整体性能。
- 3) 如果频繁发生，可能导致数据库不可用。

举例：

- **热点堵塞：**系统会突然变得非常慢，因为所有的请求都集中在一个地方（相当于所有人都跑去仓库取明星签名商品）。
- **数据库负载增加：**数据库压力增大，响应变慢。

3、预防和补救措施

- 1) **热点数据永不过期：**对于一些极为重要的热点数据，设置其缓存永不过期，或者使用定期更新缓存的机制，而不是依赖过期时间。
- 2) **互斥锁机制：**在缓存失效时，使用互斥锁（如 Redis 的分布式锁）来控制只有一个线程可以从数据库中加载数据并更新缓存，其他线程等待锁释放后从缓存中获取数据。
- 3) **缓存预加载：**对于已知的热点数据，在缓存失效前预先加载新的缓存数据，确保热点数据始终在缓存中。
- 4) **请求分摊：**使用请求分摊技术（如信号量、限流器），控制对数据库的请求量，避免大量请求同时涌入数据库。
- 5) **缓存穿透保护：**在缓存中加入空值（Null Object Pattern）保护，当查询到的数据不存在时，将其结果缓存一段时间，避免频繁查询数据库。

以下是预防和补救措施

- 1) **热点数据永不过期：**对于特别热门的商品，设置它们的缓存永不过期，或者定期刷新而不是依赖过期时间。
- 2) **互斥锁机制：**当缓存失效时，只有一个顾客可以去仓库取货，其他人等他回来再拿（使用分布式锁）。
- 3) **缓存预加载：**在商品即将下架前，提前备货（预先加载新的缓存数据）。
- 4) **请求分摊：**控制一次允许多少人去仓库取货，避免太多人同时去。
- 5) **缓存穿透保护：**如果某个商品根本不存在，把这个信息也缓存起来，避免重

复查询（防止频繁请求不存在的数据）。

理解场景：

假设超市里有一个明星签名商品，非常受欢迎，但库存有限。突然这个商品下架了，所有粉丝都跑去仓库取这个商品，导致仓库被挤爆，速度变得非常慢。为了避免这种情况，你可以确保这个明星签名商品永不过期，或者定期刷新库存。你还可以设置一个机制，让只有一个粉丝去仓库取货，其他人等他回来再拿，避免仓库被挤爆。

总结：

通过合理的缓存设计和预防措施，可以有效避免缓存雪崩和缓存击穿对系统的影响。设置不同的缓存过期时间、使用多级缓存、限流与降级策略、缓存预热和集群化部署可以预防缓存雪崩。而热点数据永不过期、互斥锁机制、缓存预加载、请求分摊和缓存穿透保护则是防止缓存击穿的有效方法。这些措施可以确保系统在高并发和大规模数据访问情况下，仍然保持高性能和高可用性。

三、 实现一个分布式缓存

要在 3 台 Linux 服务器上实现一个分布式缓存系统，我们可以使用 Redis，这是一个常用且高效的分布式缓存解决方案。下面将详细介绍实现步骤及背后的原因。

安装 Redis

首先，在每台服务器上安装 Redis。

```
# 更新包列表
sudo apt-get update

# 安装Redis
sudo apt-get install redis-server
```


配置 Redis 为集群模式

编辑每台服务器上的 Redis 配置文件，使其支持集群模式。Redis 配置文件通常位于 `/etc/redis/redis.conf`。

```
sudo nano /etc/redis/redis.conf
```

修改或添加以下配置：

```
# 开启集群模式
cluster-enabled yes

# 指定集群配置文件名称
cluster-config-file nodes.conf

# 集群节点超时时间
cluster-node-timeout 5000

# 绑定IP地址
bind your_server_ip

# 开放集群端口
port 7000
cluster-announce-port 7000
cluster-announce-bus-port 17000
```

解释：

- `cluster-enabled yes`：开启 Redis 集群模式。
- `cluster-config-file nodes.conf`：指定集群配置文件，Redis 会自动生成并管理。
- `cluster-node-timeout 5000`：节点超时时间（毫秒）。
- `bind your_server_ip`：绑定服务器的 IP 地址。
- `port 7000`：Redis 服务端口，`cluster-announce-port` 和 `cluster-announce-`

`bus-port` 指定集群通讯端口。

3. 启动 Redis 实例

在每台服务器上启动多个 Redis 实例，以实现高可用性和负载均衡。我们将在每台服务器上启动两个实例，一个作为主节点，另一个作为从节点。

```
# 创建实例目录
sudo mkdir -p /var/redis/7000
sudo mkdir -p /var/redis/7001

# 复制默认配置文件
sudo cp /etc/redis/redis.conf /var/redis/7000/
sudo cp /etc/redis/redis.conf /var/redis/7001/

# 修改端口配置
sudo nano /var/redis/7000/redis.conf
# 修改为 port 7000, dir /var/redis/7000, cluster-config-file nodes-7000.conf

sudo nano /var/redis/7001/redis.conf
# 修改为 port 7001, dir /var/redis/7001, cluster-config-file nodes-7001.conf

# 启动实例
sudo redis-server /var/redis/7000/redis.conf
sudo redis-server /var/redis/7001/redis.conf
```

解释：

- 创建实例目录和配置文件是为了在同一台服务器上运行多个 Redis 实例，便于集群配置和管理。

4. 配置 Redis 集群

使用 `redis-cli` 命令行工具配置 Redis 集群。我们需要将所有节点连接起来，并创建主从关系。

```
# 在其中一台服务器上执行
redis-cli --cluster create \
<server1_ip>:7000 <server1_ip>:7001 \
<server2_ip>:7000 <server2_ip>:7001 \
<server3_ip>:7000 <server3_ip>:7001 \
--cluster-replicas 1
```

解释:

- `redis-cli --cluster create`: 创建 Redis 集群。
- `<server1_ip>:7000 <server1_ip>:7001` 等: 指定各服务器上的 Redis 实例。
- `--cluster-replicas 1`: 设置每个主节点有一个从节点, 提高数据的可用性和容错性。

5. 验证 Redis 集群

在任意一台服务器上, 通过 `redis-cli` 连接到集群并执行测试命令:

```
redis-cli -c -p 7000
```

执行一些读写操作以确保集群正常工作:

```
SET key1 "value1"
GET key1
```

解释:

- `redis-cli -c -p 7000`: 连接到 Redis 集群模式的一个节点。
- `SET key1 "value1"` 和 `GET key1`: 测试读写操作, 以确保数据正确存储和读取。

为什么这样实现

- 1) **高可用性**: 每个主节点都有一个从节点, 确保即使某个节点失败, 数据仍然可用。
- 2) **负载均衡**: 多个主节点分担读写请求, 避免单节点瓶颈, 提高系统性能。
- 3) **数据分区**: Redis 集群模式通过分片存储数据, 每个节点只存储部分数据,

提高存储和处理能力。

- 4) **易于扩展：**可以方便地添加或移除节点，以适应业务增长或调整。

通过上述步骤，我们在 3 台 Linux 服务器上成功搭建了一个高效、可扩展的分布式缓存系统。这种架构能够满足高并发、高性能的需求，并提供良好的容错能力。